Rapport d'analyse statique - UnCrackable-Level1

Informations générales

- *Date d'analyse : 23 Avril 2026*
- *Analyste : Sultan Zeineddine*
- *APK analysé : UnCrackable-Level1.apk*
- *Version : 1.0 (OWASP MSTG)*
- *Provenance : OWASP Mobile Security Testing Guide (Crackmes)*
- *Outils utilisés : JADX GUI, dex2jar, JD-GUI, CyberChef*

Résumé exécutif

Cette analyse statique a révélé 3 vulnérabilités et faiblesses potentielles majeures dans l'application UnCrackable-Level1. Les principales préoccupations concernent l'intégration de secrets en clair dans le code source (hardcoded secrets), une logique de validation d'authentification effectuée exclusivement côté client, et l'utilisation d'un mode de chiffrement obsolète (AES-ECB). Le niveau de risque global est évalué comme Élevé, car un attaquant peut facilement extraire le mot de passe secret sans avoir besoin d'interagir avec un serveur.

Actions prioritaires recommandées :

1. *Déplacer la logique de validation du mot de passe vers un serveur distant sécurisé (Backend).*
2. *Supprimer la clé cryptographique statique et le texte chiffré du code source de l'application.*
3. *Remplacer le mode de chiffrement AES-ECB par un standard plus sécurisé comme AES-GCM.*

Constats détaillés

Constat 1 : Présence de secrets en clair dans le code (Hardcoded Secrets)

Sévérité : Élevée Description : Une clé de chiffrement AES de 128 bits au format hexadécimal (8d127684cbc37c17616d806cf50473cc) ainsi que le message secret chiffré en Base64 (5UJiFctbmgbDoLXmpL12mkno8HT4Lv8dlat8FxR2GOc=) sont codés en dur dans l'application. Localisation : Classe sg.vantagepoint.uncrackable1.a, méthodes a(String str) et b(String str). Impact potentiel : Un attaquant utilisant des outils de rétro-ingénierie (comme JADX) peut lire ces valeurs et déchiffrer le secret hors-ligne pour compromettre le mécanisme de protection de l'application. Remédiation recommandée : Ne jamais stocker de clés cryptographiques ou de données confidentielles directement dans le code source ou les ressources de l'application. Utiliser le système Android Keystore pour la gestion sécurisée des clés matérielles.

Constat 2 : Validation de sécurité effectuée côté client

Sévérité : Élevée Description : La vérification du secret saisi par l'utilisateur se fait localement sur l'appareil. Le texte saisi par l'utilisateur est comparé au secret déchiffré à la volée par l'application. Localisation : Classe sg.vantagepoint.uncrackable1.MainActivity, méthode verify(View view). Impact potentiel : Un attaquant peut manipuler le comportement de l'application (par exemple, en utilisant un

outil d'instrumentation dynamique comme Frida) pour forcer la méthode verify à toujours renvoyer true ou extraire la chaîne comparée au moment de l'exécution, contournant ainsi totalement l'authentification. Remédiation recommandée : La validation d'un accès, d'une licence ou d'un secret doit systématiquement être déléguée à un serveur distant de confiance via une API sécurisée (HTTPS). L'application mobile ne doit être qu'une interface d'affichage.

Constat 3 : Utilisation d'un mode de chiffrement cryptographique faible

Sévérité : Moyenne Description : L'algorithme de chiffrement implémenté pour cacher le secret utilise AES en mode ECB (Electronic Codebook) avec un remplissage PKCS7 (AES/ECB/PKCS7Padding). Localisation : Classe sg.vantagepoint.a.a, méthode a(byte[] bArr, byte[] bArr2). Impact potentiel : Le mode ECB n'utilise pas de vecteur d'initialisation (IV). Par conséquent, des blocs de texte clair identiques produiront toujours des blocs de texte chiffré identiques. Bien que l'impact soit limité ici car le secret est court, ce mode est considéré comme cryptographiquement non sécurisé pour les données modernes. Remédiation recommandée : Remplacer le mode ECB par des modes de chiffrement authentifiés modernes et robustes tels que AES-GCM (Galois/Counter Mode) ou, à défaut, AES-CBC avec un vecteur d'initialisation (IV) généré aléatoirement.

Annexes

Permissions demandées

- *L'analyse de l'application ne révèle aucune permission excessive dans le manifeste. L'application ne demande pas d'accès réseau (INTERNET) ni d'accès au stockage.*

Composants exportés

- *sg.vantagepoint.uncrackable1.MainActivity (Exporté par défaut car il contient un intent-filter avec l'action android.intent.action.MAIN et la catégorie android.intent.category.LAUNCHER, ce qui est un comportement normal pour le point d'entrée d'une application).*

Autres éléments pertinents

- *Mécanismes de défense découverts : L'application intègre des vérifications basiques de détection d'environnement Root (classe sg.vantagepoint.a.c) et de débogage (classe sg.vantagepoint.a.b). Si l'une de ces conditions est remplie, la méthode onCreate déclenche un System.exit(0). Ces mesures compliquent l'analyse dynamique mais ne protègent pas contre l'analyse statique.*